

Продолжаем изучать маршрутизацию

В ASP NET Core

Ограничения маршрутов

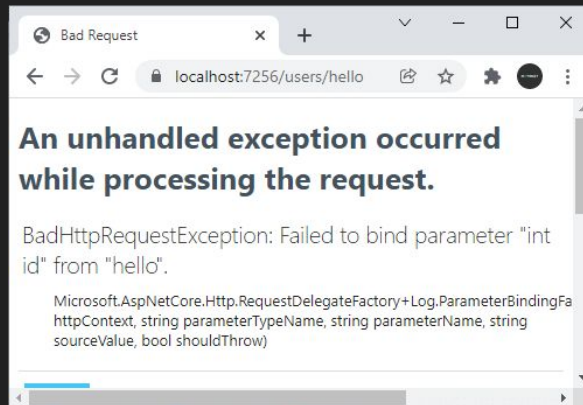
При обработке запроса система маршрутизации на основании шаблон маршрута автоматически извлекает из строки запроса значения для параметров маршрута вне зависимости от содержимого этих значений. Однако это не всегда бывает удобно. Например, мы хотим, чтобы какой-то параметр представлял только числа, а другой параметр начинался строго с определенного символа. И для этого необходимо задать **ограничения маршрута (route constraints)**.

Ограничения маршрутов выполняются при парсинге пути запроса, сопоставлении его с шаблоном маршрута и выделении из него значений для параметров маршрута.

Ограничения маршрутов применяются для разных задач. Прежде всего они решают, допустимо ли для параметра маршрута значение, которое выделено из пути запроса. Также ограничения маршрута могут решать, можно ли вообще сопоставить путь запроса с определенным маршрутом. Кроме того, ограничения маршрута могут применяться при генерации ссылок.

Вначале рассмотрим следующую ситуацию. Пусть у нас есть следующее приложение:

```
var builder =  
    WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Map("/users/{id}", (int id) => $"User  
Id: {id}");  
app.Map("/", () => "Index Page");  
  
app.Run();
```



Ограничения маршрутов

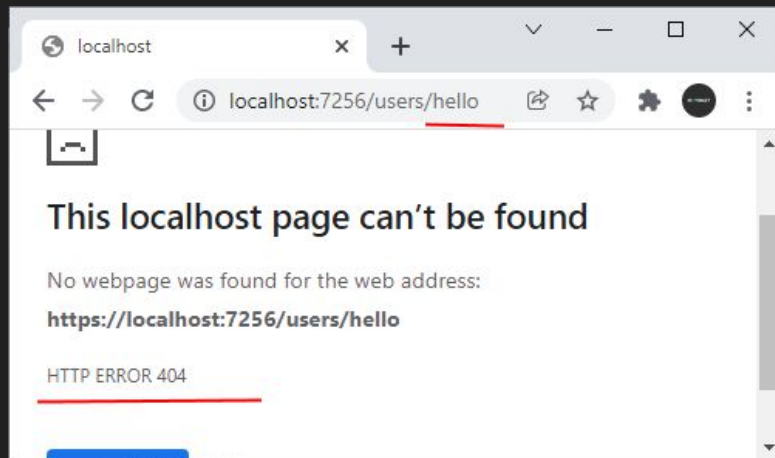
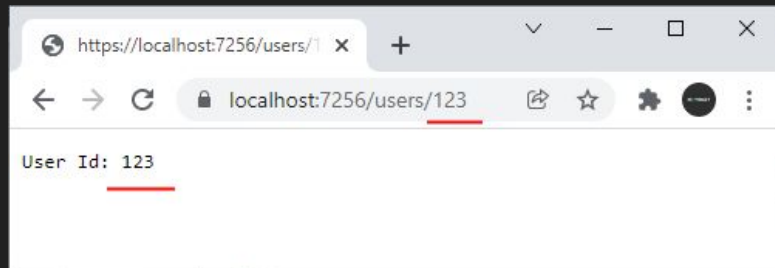
Теперь применим ограничения - укажем, что параметр `id` в маршруте должен представлять тип `int`:

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();
```

```
app.Map("/users/{id:int}", (int id) => $"User  
Id: {id}");
```

```
app.Map("/", () => "Index Page");
```

```
app.Run();
```



Ограничения маршрутов

Ограничения можно комбинировать. При применении нескольких ограничений одновременно, они отделяются друг от друга двоеточием. Например:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

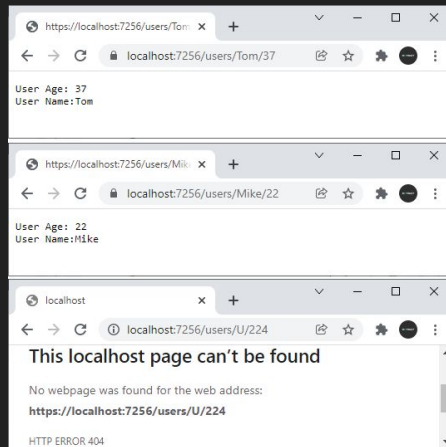
app.Map(
    "/users/{name:alpha:minlength(2)}/{age:int:range(1, 110)}",
    (string name, int age) => $"User Age: {age} \nUser Name:{name}"
);
app.Map(
    "/phonebook/{phone:regex(^7-\\d{3}-\\d{3}-\\d{4}$)}/",
    (string phone) => $"Phone: {phone}"
);
app.Map("/", () => "Index Page");
```

```
app.Run();
```

Первая конечная точка использует шаблон маршрута

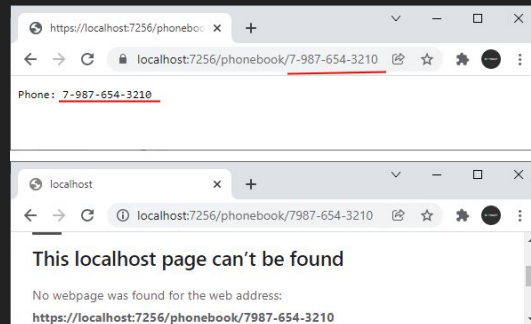
```
"/users/{name:alpha:minlength(2)}/{age:int:range(1, 110)}"
```

Здесь предполагается, что параметр name принимает только алфавитные символы, а его минимальная длина должна представлять два символа. Второй же параметр маршрута - age должен представлять целое число и должен находиться в диапазоне между 1 и 110:



Вторая конечная точка использует шаблон маршрута
`"/phonebook/{phone:regex(^7-\\d{3}-\\d{3}-\\d{4}$)}/"`

Параметр phone принимает номер телефона в формате 7-xxx-xxx-xxxx, любые другие форматы будут некорректными:



Создание ограничений маршрутов

Хотя фреймворк ASP.NET Core по умолчанию предоставляет большой набор встроенных ограничений, их может быть недостаточно. И в этом случае мы можем определить свои кастомные ограничения маршрутов.

Для создания собственного ограничения маршрута нужно реализовать интерфейс **IRouteConstraint** с одним единственным методом **Match**, который имеет следующее определение:

```
public interface IRouteConstraint
{
    bool Match(HttpContext? httpContext,
               IRouter? route,
               string routeKey,
               RouteValueDictionary values,
               RouteDirection routeDirection);
}
```

Параметры метода:

- `httpContext`: объект **HttpContext**, который инкапсулирует информацию о HTTP-запросе
- `route`: объект **IRouter**, который представляет маршрут, в рамках которого применяется ограничение
- `routeKey`: объект **String** - название параметра маршрута, к которому применяется ограничение
- `values`: объект **RouteValueDictionary**, который представляет набор параметров маршрута в виде словаря, где ключи - названия параметров, а значения - значения параметров маршрута
- `routeDirection`: объект перечисления **RouteDirection**, которое указывает, применяется ограничение при обработке запроса, либо при генерации ссылки

В качестве результата метод возвращает значение типа **bool**: **true**, если запрос удовлетворяет данному ограничению маршрута, и **false**, если не удовлетворяет.

Ограничение маршрута применяет этот интерфейс **IRouteConstraint**. Это вынуждает движок маршрутизации вызвать для ограничения маршрута метод `IRouteConstraint.Match`, чтобы определить, применяется ли данное ограничение к данному запросу или нет. И только если данный метод возвращает **true**, запрос может быть сопоставлен с маршрутом (конечно, если запрос также удовлетворяет и другим ограничениям, которые могут применяться).

Ограничение для параметра маршрута

Допустим, клиент в запросе через параметр должен передавать код, который равен некоторой строке. Для этого определим следующий класс ограничения маршрута:

```
public class SecretCodeConstraint : IRouteConstraint
{
    string secretCode; // допустимый код
    public SecretCodeConstraint(string secretCode)
    {
        this.secretCode = secretCode;
    }

    public bool Match(HttpContext? httpContext, IRouter? route, string routeKey, RouteValueDictionary values, RouteDirection routeDirection)
    {
        return values[routeKey]?.ToString() == secretCode;
    }
}
```

Класс `SecretCodeConstraint` через конструктор принимает некий условно секретный код, которому должен соответствовать параметр маршрута. В методе **Match()** с помощью выражения `values[routeKey]` получаем из словаря `values` значение параметра маршрута, имя которого передается через `routeKey`. И затем это значение сравниваем с секретным кодом:

```
return values[routeKey]?.ToString() == secretCode;
```

Если оба значения равны, то возвращаем `true`, что указывает, что маршрут соответствует данному ограничению.

Ограничение для параметра маршрута

Применим данное ограничение:

```
var builder = WebApplication.CreateBuilder();
// проецируем класс SecretCodeConstraint на inline-ограничение secretcode
builder.Services.Configure<RouteOptions>(options =>
    options.ConstraintMap.Add("secretcode", typeof(SecretCodeConstraint)));

// альтернативное добавление класса ограничения
// builder.Services.AddRouting(options => options.ConstraintMap.Add("secretcode", typeof(SecretCodeConstraint)));

var app = builder.Build();

app.Map(
    "/users/{name}/{token:secretcode(123466)}/",
    (string name, int token) => $"Name: {name} \nToken: {token}"
);
app.Map("/", () => "Index Page");

app.Run();

public class SecretCodeConstraint : IRouteConstraint
{
    string secretCode; // допустимый код
    public SecretCodeConstraint(string secretCode)
    {
        this.secretCode = secretCode;
    }

    public bool Match(HttpContext? httpContext, IRouter? route, string routeKey, RouteValueDictionary values, RouteDirection routeDirection)
    {
        return values[routeKey]?.ToString() == secretCode;
    }
}
```

Здесь надо отметить следующий момент. Если мы хотим использовать класс ограничения как inline-ограничение внутри шаблона маршрута, то нам необходимо изменить настройки для сервиса **RouteOptions**:

```
builder.Services.Configure<RouteOptions>(options =>
    options.ConstraintMap.Add("secretcode",
        typeof(SecretCodeConstraint)));
```

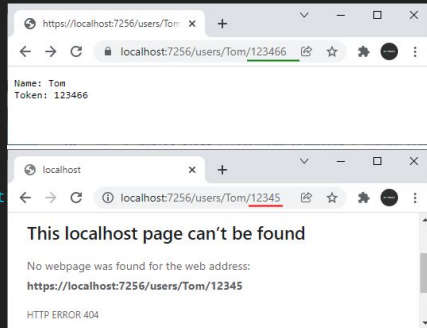
В данном случае параметр options представляет объект RouteOptions. Его свойство **ConstraintMap** представляет коллекцию применяемых ограничений. Метод **Add()** добавляет в эту коллекцию ограничение. Причем первый параметр этого метода представляет ключ ограничения в коллекции ограничений, а второй параметр - собственно класс ограничения. Далее ключ ограничения затем можно будет применять как inline-ограничение, на которое проецируется класс ограничения.

В качестве альтернативы вместо этого вызова мы могли бы обратиться к сервисам маршрутизации и также настроить объект RouteOptions:

```
builder.Services.AddRouting(options =>
    options.ConstraintMap.Add("secretcode",
        typeof(SecretCodeConstraint)));
```

После этого мы сможем использовать inline-ограничение в шаблоне маршрута: `"/users/{name}/{token:secretcode(123466)}/"`

То есть здесь к параметру маршрута token будет применяться ограничение SecretCodeConstraint. А строка 123466 - это тот секретный код, который будет передаваться в конструктор объекта SecretCodeConstraint.



Ограничение для параметра маршрута

Другой пример. Допустим, мы хотим, чтобы параметр не мог иметь одно из набора значений:

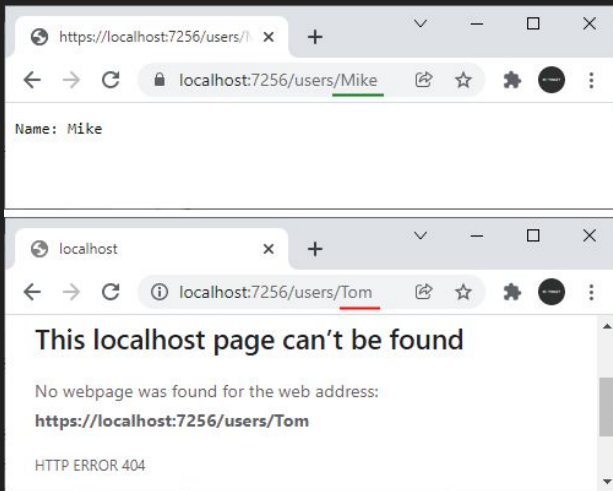
```
var builder = WebApplication.CreateBuilder();
builder.Services.AddRouting(options =>
    options.ConstraintMap.Add("invalidnames", typeof(InvalidNamesConstraint)));
var app = builder.Build();

app.Map("/users/{name:invalidnames}", (string name) => $"Name: {name}");
app.Map("/", () => "Index Page");

app.Run();

public class InvalidNamesConstraint : IRouteConstraint
{
    string[] names = new[] { "Tom", "Sam", "Bob" };
    public bool Match(HttpContext? httpContext, IRouter? route, string routeKey,
        RouteValueDictionary values, RouteDirection routeDirection)
    {
        return !names.Contains(values[routeKey]?.ToString());
    }
}
```

В данном случае параметр маршрута name НЕ должен представлять имена "Tom", "Sam" и "Bob":



Передача зависимостей в конечные точки

Фреймворк ASP.NET Core предоставляет простой и удобный способ для передачи зависимостей в конечные точки. Все добавляемые в коллекцию сервисов приложения зависимости можно получить через параметры делегата, который отвечает за обработку запроса.

Например, определим следующее приложение:

```
var builder = WebApplication.CreateBuilder();

builder.Services.AddTransient<TimeService>(); // Добавляем сервис

var app = builder.Build();

app.Map("/time", (TimeService timeService) => $"Time: {timeService.Time}");
app.Map("/", () => "Hello World");

app.Run();

// сервис
public class TimeService
{
    public string Time => DateTime.Now.ToLongTimeString();
}
```

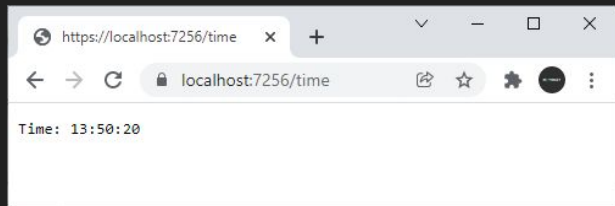
Здесь класс `TimeService` выступает в качестве сервиса, свойство `Time` которого возвращает текущее время в формате "hh:mm:ss".

Этот сервис добавляется в коллекцию сервисов приложения:

```
builder.Services.AddTransient<TimeService>();
```

Далее через параметр делегата, который передается в качестве второго параметра в метод `Map()` мы можем получить эту зависимость:

```
app.Map("/time", (TimeService timeService) =>
    $"Time: {timeService.Time}");
```



Передача зависимостей в конечные точки

Подобным образом можно получить зависимости, если обработчик маршрута конечной точки вынесен в отдельный метод:

```
var builder = WebApplication.CreateBuilder();

builder.Services.AddTransient<TimeService>();    // Добавляем сервис

var app = builder.Build();

app.Map("/time", SendTime);
app.Map("/", () => "Hello World");

app.Run();

string SendTime(TimeService timeService)
{
    return $"Time: {timeService.Time}";
}

public class TimeService
{
    public string Time => DateTime.Now.ToLongTimeString();
}
```

Сопоставление запроса с конечной точкой

Сопоставление адреса URL или **URL matching** представляет процесс сопоставления запроса с конечной точкой. Данный процесс основывается на пути запроса и полученных в запросе заголовках. Данный процесс проходит ряд этапов:

1. Сначала выбираются все конечные точки, шаблон маршрута которых совпадает с путем запроса
2. Далее из полученного на предыдущем этапе набора конечных точек удаляются те, которые не соответствуют ограничениям маршрута
3. Затем из полученного на предыдущем этапе набора конечных точек удаляются те, которые не удовлетворяют политике объекта **MatcherPolicy** (вкратце: класс **MatcherPolicy** позволяет определить порядок сравнения конечных точек и адреса URL)
4. И в самом конце применяется объект **EndpointSelector** для выбора из полученного на предыдущем этапе списка конечной точки, которая в конечном счете и будет обрабатывать запрос

Приоритет конечных точек зависит от двух факторов:

- Порядок следования в наборе конечных точек
- Приоритетность шаблона маршрута

Приоритетность шаблонов маршрута зависит от специфичности шаблона. Специфичность шаблона определяется на основе следующих критериев:

- Шаблон маршрута с большим количеством сегментов более специфичен, чем шаблон меньшим количеством сегментов
- Сегмент с текстовым литералом (статический сегмент) более специфичен, чем сегмент с параметром маршрута
- Сегмент с параметром, к которому применяется ограничение маршрута, более специфичен, чем сегмент с параметром без ограничения
- Комплексный сегмент более специфичен, чем сегмент с параметром с ограничением
- Параметр catch-all (параметр, который соответствует неопределенному количеству сегментов) наименее специфичен

Если в конечном счете осталось две и более конечных точек, которые соответствуют запрошенному адресу, и соответственно система маршрутизации не может выбрать, какая из этих конечных точек должна обрабатывать маршрут, то генерируется исключение.

Сопоставление запроса с конечной точкой

Например, пусть у нас есть два шаблона маршрута: `"/hello"` и `"/{message}"`.

```
var builder = WebApplication.CreateBuilder();

var app = builder.Build();

app.Map("/hello", () => "Hello World");
app.Map("/{message}", (string message) => $"Message: {message}");

app.Map("/", () => "Index Page");

app.Run();
```

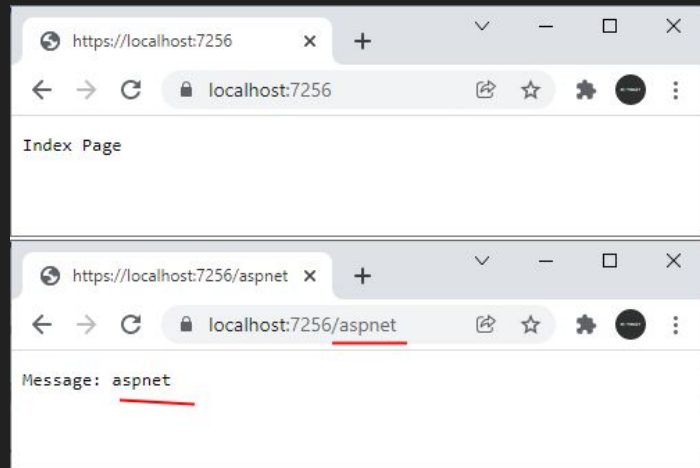
Оба этих маршрута соответствуют пути запроса `"/hello"`. Однако шаблон маршрута `"/{message}"` более общий - вместо параметра `message` может идти что угодно. Тогда как шаблон `"/hello"` состоит из статического сегмента и соответственно более конкретный, более специфичный, поэтому конечная точка этого шаблона маршрута и будет выбрана для обработки запроса по пути `"/hello"`.

Сопоставление запроса с конечной точкой

Другой пример:

```
var builder =  
    WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Map("/{message?}", (string? message) =>  
    $"Message: {message}");  
app.Map("/", () => "Index Page");  
  
app.Run();
```

Здесь первый шаблон маршрута применяет необязательный параметр `message`. Второй шаблон маршрута соответствует корню веб-приложения. И в принципе оба этих шаблона соответствуют пути запроса `/`. Однако второй шаблон представляет статический сегмент, поэтому его конечная точка будет выбрана в конечном счете для обработки запроса:



Сопоставление запроса с конечной точкой

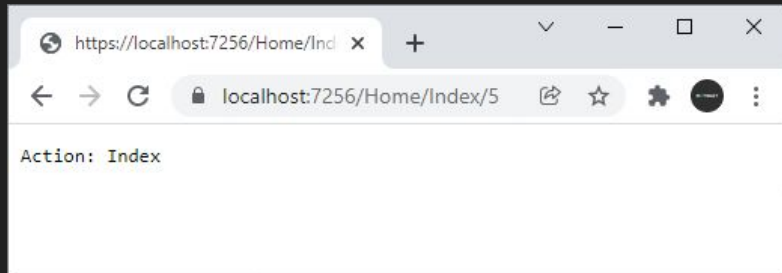
Более сложный пример:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/{controller}/Index/5", (string
controller) => $"Controller: {controller}");
app.Map("/Home/{action}/{id}", (string action)
=> $"Action: {action}");

app.Run();
```

Здесь опять же поскольку во втором маршруте первый сегмент представляет статический сегмент, то именно вторая конечная точка будет выбираться для обработки маршрута:



Сочетание конечных точек с другими middleware

Кроме конечных точек запрос в конвейере обработки могут обрабатывать и другие компоненты middleware. При этом надо учитывать общий процесс обработки запроса и вызова конечных точек.

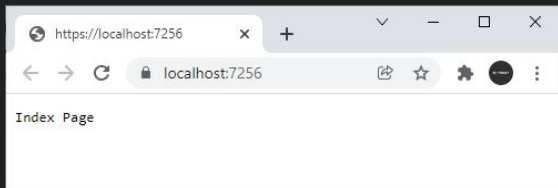
Так, если приложение содержит конечные точки, то система маршрутизации на основе процессе URL matching или сопоставления адреса URL с шаблонами маршрута выбирает для обработки определенную конечную точку. Если в приложении есть такая конечная точка, которая соответствует запросу, то компонент middleware

Microsoft.AspNetCore.Routing.EndpointRoutingMiddleware устанавливает у объекта `HttpContext` конечную точку для будущей обработки запроса, которую можно получить с помощью метода `HttpContext.GetEndpoint()`. Кроме того, устанавливаются значения маршрута, которые можно получить через коллекцию `HttpRequest.RouteValues`

Однако конечная точка начинает обрабатывать запрос только после того, как все middleware в конвейере начнут обработку запроса. Например, возьмем следующий код приложения:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Use(async (context, next) =>
{
    Console.WriteLine("First middleware starts");
    await next.Invoke();
    Console.WriteLine("First middleware ends");
});
app.Map("/", () =>
{
    Console.WriteLine("Index endpoint starts and ends");
    return "Index Page";
});
app.Use(async (context, next) =>
{
    Console.WriteLine("Second middleware starts");
    await next.Invoke();
    Console.WriteLine("Second middleware ends");
});
app.Map("/about", () =>
{
    Console.WriteLine("About endpoint starts and ends");
    return "About Page";
});
app.Run();
```



Сочетание конечных точек с другими middleware

в компонентах middleware также можно обрабатывать запросы по определенным адресам:

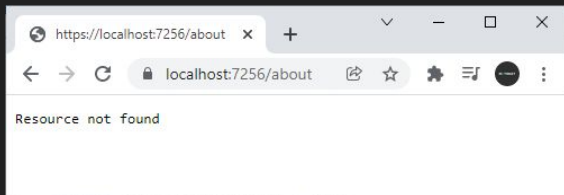
```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Use(async (context, next) =>
{
    if (context.Request.Path == "/date")
        await context.Response.WriteAsync($"Date: {DateTime.Now.ToShortDateString()}");
    else
        await next.Invoke();
});

app.Map("/", () => "Index Page");
app.Map("/about", () => "About Page");

app.Run();
```

Кроме того, middleware могут быть полезны для некоторого постдействия - выполнения некоторых действий, когда конечная точка уже выбрана. Или, наоборот, если ни одна из конечных точек не обработала запрос, и в middleware мы можем обработать эту ситуацию:



```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Use(async (context, next) =>
{
    await next.Invoke();

    if (context.Response.StatusCode == 404)
        await context.Response.WriteAsync("Resource Not Found");
});
```

Однако если в конце конвейера располагается терминальный компонент, то он будет выполняться даже если конечная точка соответствует запрошенному пути:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/", () => "Index Page");
app.Map("/about", () => "About Page");

app.Run(async context =>
{
    context.Response.StatusCode = 404;
    await context.Response.WriteAsync("Resource not found");
});

app.Run();
```


Получение параметров строки запроса

Одной из форм передачи данных приложению на ASP.NET представляет передача строки запроса, то есть ту часть адреса URL, которая идет после символа ? (вопросительного знака). Например:

`https://localhost:7206/user?name=Tom&age=39`

В данном случае часть `?name=Tom&age=39` представляет строку запроса.

Строка запроса состоит из параметров в виде

`?параметр1&параметр2&параметрN`

Каждый параметр отделяется от другого с помощью символа амперсанда & и определяется в следующем виде:

`имя_параметра=значение_параметра`

После имени параметра через символ равно указывается значение параметра. Например, в строке запроса из следующего адреса:

`https://localhost:7206/user?name=Tom&age=39`

определены два параметра. Первый параметр называется "name" и имеет значение "Tom", а второй параметр называется "age" и имеет значение 39.

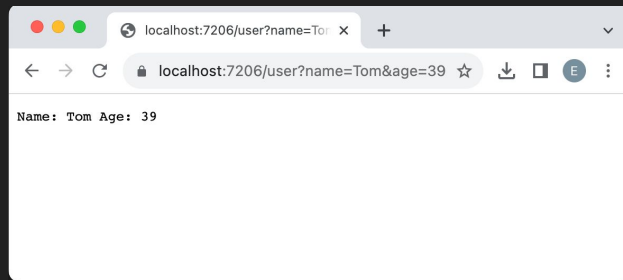
Для получения параметров строки запроса в конечной точке ASP.NET Core надо определить обработчик, параметры которого соответствуют названиям параметров. Например, возьмем следующее приложение:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
```

```
app.MapGet("/user", (string name, int age) => $"Name: {name} Age: {age}");
```

```
app.MapGet("/", () => "Hello World");
app.Run();
```

Здесь для первой конечной точки, которая обрабатывает запросы по адресу "/user", определен обработчик, который принимает два параметра - name и age. Значения этих параметров приложение будет брать из строки запроса. Например, если мы обратимся с запросом /user?name=Tom&age=39, то строка "Tom" перейдет в параметр name, а число 39 - в параметр age:



Получение параметров строки запроса

Стоит отметить, что между параметрами в приложении ASP.NET и значениями параметров строки запроса должно быть соответствие по типу данных. Так, в примере выше параметр `age` представляет тип `int`, соответственно ему надо передать целочисленное значение. Иначе ASP.NET не сможет связать значение с параметром `age`.

В примере выше оба параметра являются обязательными, нам обязательно надо передать для них значения. Если мы хотим сделать параметр необязательным, то в качестве типа можно использовать соответствующий nullable-тип:

```
var builder = WebApplication.CreateBuilder(args);  
var app = builder.Build();  
  
app.MapGet("/user", (string name, int? age) => $"Name: {name} Age: {age??0}");  
  
app.MapGet("/", () => "Hello World");  
app.Run();
```

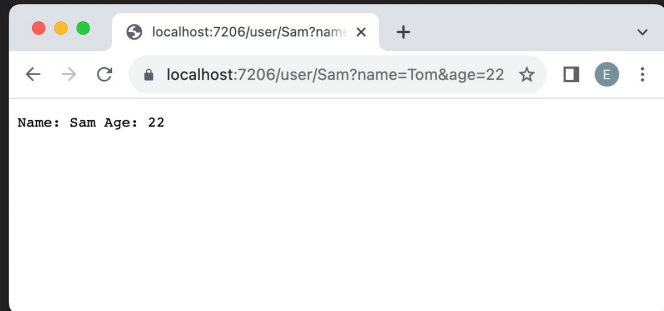
Здесь параметр `age` является необязательным и, если он не указан, например, в запросе `/user?name=Sam`, то параметр получает значение `null`.

Получение параметров строки запроса

Стоит отметить, что одна конечная точка может сочетать одноименные параметры маршрута и строки запроса, в этом случае параметры маршрута будут иметь приоритет:

```
var builder = WebApplication.CreateBuilder(args);  
var app = builder.Build();  
  
app.MapGet("/user/{name}", (string name, int age) =>  
    $"Name: {name} Age: {age}");  
  
app.MapGet("/", () => "Hello World");  
app.Run();
```

Здесь, для первой конечной точки определен параметр маршрута `name`. И даже если мы передадим параметр строки запроса, который называется `name`, то обработчик конечной точки все равно получит данные именно из параметра маршрута:



Объединение параметров

Если параметров строки запроса несколько, то с ними менее удобно работать, и, возможно, мы захотим объединить их в какой-нибудь общий тип данных, наподобие следующего:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/user", (Person person) => $"Name: {person.Name} Age: {person.Age}");
app.MapGet("/", () => "Hello World");

app.Run();

record Person(string Name, int Age);
```

Здесь два параметра name и age объединены в один тип Person, и мы ожидаем, что при передаче строки запроса с параметрами name и age среда ASP.NET сможет связать их со свойствами объекта Person. Однако этот способ не работает, поскольку по умолчанию ASP.NET Core будет искать значения для объектов не примитивных типов в теле запроса, а не в строке запроса. Соответственно при выполнении мы получим ошибку. Чтобы явным образом указать, что данные будут браться из строки запроса, надо использовать атрибут **[AsParameters]**:

```
app.MapGet("/user", ([AsParameters] Person person) => $"Name: {person.Name} Age: {person.Age}");
```

Статические файлы. Установка каталога статических файлов. UseStaticFiles

Ранее уже рассматривалась отправка статических файлов. В частности, для отправки файлов мы могли использовать метод `SendFileAsync()` объекта `HttpResponse`:

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Run(async (context) => await context.Response.SendFileAsync("index.html"));  
  
app.Run();
```

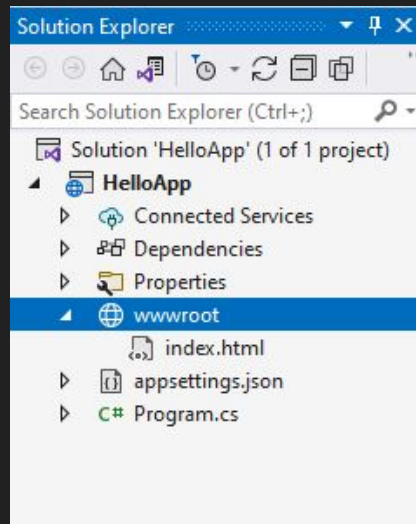
Установка каталога статических файлов. UseStaticFiles

Однако ASP.NET Core также предоставляет специальный встроенный middleware, который подключается с помощью метода **UseStaticFiles()** и который упрощает работу со статическими файлами.

При использовании этого middleware применяются некоторые условности. В частности, по умолчанию для определения пути хранения статических файлов в проекте используются два параметра **ContentRoot** и **WebRoot**, а сами статические файлы должны помещаться в каталог ContentRoot/WebRoot. На стадии разработки параметр "ContentRoot" соответствует каталогу текущего проекта. А параметр "WebRoot" по умолчанию представляет папку **wwwroot** в рамках каталога ContentRoot. То есть, исходя из значений по умолчанию, то статические файлы следует располагать в папке "wwwroot", которая должна находиться в текущем проекте. Однако эти параметры при необходимости можно переопределить.

В разных типах проектов ASP NET Core данная папка может уже быть по умолчанию в проекте, а может отсутствовать. Например, в проекте по типу Empty данная папка отсутствует, поэтому ее надо добавлять вручную.

Итак, возьмем проект по типу Empty и добавим в него новую папку **wwwroot**. Далее добавим в папку **wwwroot** новый файл **index.html**. То есть у нас получится следующий проект:



Установка каталога статических файлов. UseStaticFiles

Изменим код файла index.html, например, следующим образом:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Test</title>
</head>
<body>
  <h1>Index Page</h1>
</body>
</html>
```

Для того, чтобы клиенты могли обращаться к этому файлу, подключим соответствующий компонент middleware с помощью метода UseStaticFiles():

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseStaticFiles();    // добавляем поддержку статических файлов

app.Run(async (context) => await context.Response.WriteAsync("Hello World"));

app.Run();
```

Установка каталога статических файлов. UseStaticFiles

Для подключения функциональности работы со статическими файлами применяется метод **UseStaticFiles()**, который реализован как метод расширения для типа `IApplicationBuilder`.

Теперь, если мы обратимся к добавленному файлу, например, по пути `/index.html`, то нам отобразится содержимое данной веб-страницы

По всем остальным запросам браузер выводил бы строку "Hello World".

Если бы `index.html` находился бы в какой-то вложенной папке, например, в `wwwroot/html/`, то для обращения к нему мы могли бы использовать путь `/html/index.html`. То есть middleware для работы со статическими сайтами автоматически сопоставляет запросы с путями к статическим файлам в рамках папки `wwwroot`.

Изменение пути к статическим файлам

Что делать, если нас не устраивает стандартная папка `wwwroot`. И мы, к примеру, хотим, чтобы все статические файлы в проекте находились в папке **static**. Для этого добавим в проект папку **static** в проект и определим в ней какой-нибудь html-файл. Пусть он будет называться **content.html**

Допустим, в этом файле будет какое-нибудь простейшее содержимое:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Test</title>
</head>
<body>
  <h1>Content from Static folder</h1>
</body>
</html>
```

Чтобы приложение восприняло эту папку, изменим код создания хоста в файле Program.cs:

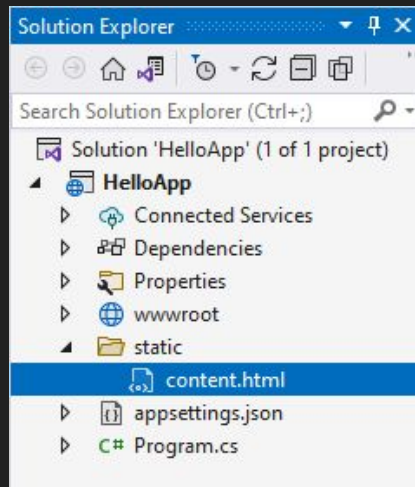
```
var builder = WebApplication.CreateBuilder(
    new WebApplicationOptions { WebRootPath = "static" }); // изменяем папку для хранения
статика

var app = builder.Build();

app.UseStaticFiles(); // добавляем поддержку статических файлов

app.Run(async (context) => await context.Response.WriteAsync("Hello World"));

app.Run();
```



Для добавления пути к файлам используется перегруженная версия метода **CreateBuilder()**, которая в качестве параметра принимает объект **WebApplicationOptions**. Его свойство **WebRootPath** позволяет установить папку для статических файлов.

И после этого мы также сможем обращаться к статическим файлам из папки **static**

Работа со статическими файлами

Рассмотрим некоторые возможности, которые предоставляет нам ASP.NET Core для работы со статическими файлами.

Файлы по умолчанию

Допустим, в проекте в папке **wwwroot** располагается файл **index.html**:

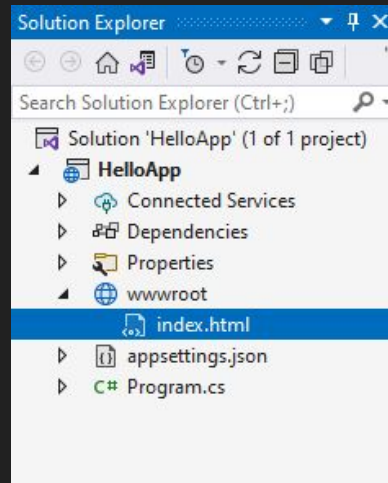
С помощью специального метода расширения **UseDefaultFiles()** можно настроить отправку статических веб-страниц по умолчанию без обращения к ним по полному пути:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseDefaultFiles(); // поддержка страниц html по умолчанию
app.UseStaticFiles();

app.Run(async (context) => await context.Response.WriteAsync("Hello
World"));

app.Run();
```



Файлы по умолчанию

В этом случае при отправке запроса к корню веб-приложения типа `http://localhost:xxxx/` приложение будет искать в папке `wwwroot` следующие файлы:

- `default.htm`
- `default.html`
- `index.htm`
- `index.html`

Если файл будет найден, то он будет отправлен в ответ клиенту.

Если же файл не будет найден, то продолжается обычная обработка запроса с помощью следующих компонентов `middleware`. То есть фактически это будет аналогично, как будто мы обращаемся к файлу: `http://localhost/index.html`

Если же мы хотим использовать файл, название которого отличается от вышеперечисленных, то нам надо в этом случае применить объект

`DefaultFileOptions`:

В этом случае в качестве страницы по умолчанию будет использоваться файл **`hello.html`**, который должен располагаться в папке `wwwroot`.

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

DefaultFileOptions options = new
DefaultFileOptions();
options.DefaultFileNames.Clear(); // удаляем имена
файлов по умолчанию
options.DefaultFileNames.Add("hello.html"); //
добавляем новое имя файла
app.UseDefaultFiles(options); // установка
параметров

app.UseStaticFiles();

app.Run(async (context) => await
context.Response.WriteAsync("Hello World"));

app.Run();
```

Метод UseDirectoryBrowser

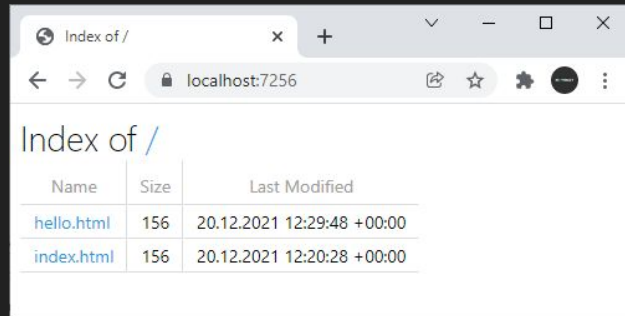
Метод **UseDirectoryBrowser** позволяет пользователям просматривать содержимое каталогов на сайте:

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.UseDirectoryBrowser();  
app.UseStaticFiles();
```

```
app.Run();
```

Данный метод имеет перегрузку, которая позволяет сопоставить определенный каталог на жестком диске или в проекте с некоторой строкой запроса и тем самым потом отобразить содержимое этого каталога:

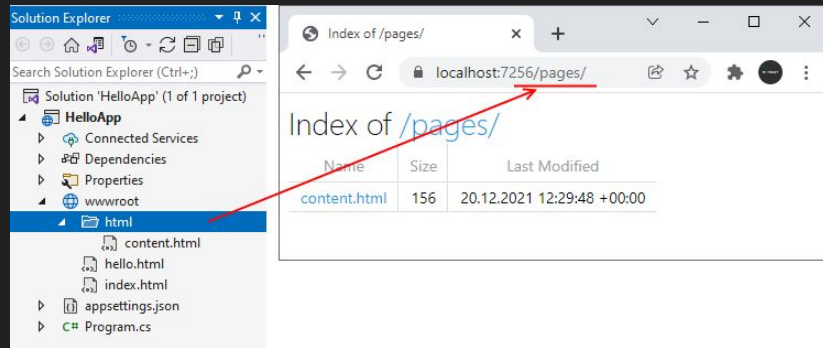
```
using Microsoft.Extensions.FileProviders;  
  
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.UseDirectoryBrowser(new DirectoryBrowserOptions()  
{  
    FileProvider = new PhysicalFileProvider(Path.Combine(Directory.GetCurrentDirectory(),  
@"wwwroot\html")),  
  
    RequestPath = new PathString("/pages")  
});  
app.UseStaticFiles();  
  
app.Run();
```



Метод UseDirectoryBrowser

Чтобы задействовать новый функционал, надо подключить пространство имен `using Microsoft.Extensions.FileProviders`.

В качестве параметра метод `UseDirectoryBrowser()` принимает объект **DirectoryBrowserOptions**, который позволяет настроить сопоставление путей к файлам с каталогами. Так, в данном случае путь типа `http://localhost:xxxx/pages/` будет сопоставляться с каталогом "wwwroot\html".



Сопоставление каталогов с путями

Перегрузка метода `UseStaticFiles()` позволяет сопоставить пути с определенными каталогами:

```
using Microsoft.Extensions.FileProviders;

var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseStaticFiles();
app.UseStaticFiles(new StaticFileOptions() // обрабатывает запросы к каталогу wwwroot/html
{
    FileProvider = new PhysicalFileProvider(
        Path.Combine(Directory.GetCurrentDirectory(), @"wwwroot\html")),
    RequestPath = new PathString("/pages")
});

app.Run();
```

Первый вызов `app.UseStaticFiles()` обрабатывает запросы к файлам в папке `wwwroot`. Второй вызов принимает те же параметры, что и метод `app.UseDirectoryBrowser()` в предыдущем примере. И в отличие от первого вызова он обрабатывает запросы по пути `http://localhost:xxxx/pages`, сопоставляя данные запросы с папкой **`wwwroot/html`**. К примеру, по запросу `http://localhost:xxxx/pages/index.html` мы можем обратиться к файлу **`wwwroot/html/index.html`**.

Метод UseFileServer

Метод `UseFileServer()` объединяет функциональность сразу всех трех вышеописанных методов `UseStaticFiles`, `UseDefaultFiles` и `UseDirectoryBrowser`:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseFileServer();
```

```
app.Run();
```

По умолчанию этот метод позволяет обрабатывать статические файлы и отправлять файлы по умолчанию типа `index.html`. Если нам надо еще включить просмотр каталогов, то мы можем использовать перегрузку данного метода:

```
app.UseFileServer(enableDirectoryBrowsing: true);
```

Еще одна перегрузка метода позволяет более точно задать параметры:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.UseFileServer(new FileServerOptions
{
    EnableDirectoryBrowsing = true,
    EnableDefaultFiles = false
});

app.Run();
```

Также можно настроить сопоставление путей запроса с каталогами:

```
using Microsoft.Extensions.FileProviders;
```

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();
```

```
app.UseFileServer(new FileServerOptions
{
    EnableDirectoryBrowsing = true,
    FileProvider = new
PhysicalFileProvider(Path.Combine(Directory.GetCurrentD
irectory(), @"wwwroot\html")),
    RequestPath = new PathString("/pages"),
    EnableDefaultFiles = false
});

app.Run();
```

В этом случае будет разрешен обзор каталога по пути `http://localhost:xxxx/pages/`, но при этом путь `http://localhost:xxxx/html/` работать не будет.

Статические файлы и MapStaticAssets

В ASP.NET Core 9.0 был добавлен новый компонент middleware - **MapStaticAssets**. Посмотрим, чем он отличается от стандартного компонента **UseStaticFiles**, который применялся в ранних версиях ASP.NET Core.

Прежде всего **UseStaticFiles** обслуживает статические файлы, но не обеспечивает тот же уровень оптимизации, что и **MapStaticAssets**. **MapStaticAssets** оптимизирован для обслуживания ресурсов, о которых приложение знает во время выполнения. Если приложение обслуживает ресурсы из других мест, таких как диск или встроенные ресурсы, следует использовать **UseStaticFiles**.

MapStaticAssets предоставляет следующие преимущества, которые недоступны при вызове **UseStaticFiles**:

- Сжатие во время сборки для всех ресурсов в приложении, включая JavaScript и CSS, но исключая ресурсы изображений и шрифтов, которые уже сжаты. Во время разработки применяется сжатие Gzip (Content-Encoding: gz), а во время публикации - сжатие Brotli (Content-Encoding: br).
- Для всех ресурсов во время сборки применяется отпечаток Fingerprinting с помощью строки хэша SHA-256 для содержимого каждого файла в кодировке Base64. Это предотвращает повторное использование старой версии файла, даже если старый файл кэширован. Статические ресурсы с отпечатком кэшируются с помощью директивы immutable (указывает, что ответ не будет обновляться, пока он актуален), что приводит к тому, что браузер никогда не запрашивает ресурс снова, пока он не изменится. Для браузеров, которые не поддерживают директиву immutable, добавляется директива max-age.
- Даже если ресурс не применяет отпечаток fingerprinting, для каждого статического ресурса генерируется тег ETag на основе контента с использованием хэша fingerprint файла в качестве значения ETag. Это гарантирует, что браузер загружает файл только в том случае, если его содержимое изменяется (или файл загружается впервые). Внутренне фреймворк сопоставляет физические ресурсы с их отпечатками fingerprint, что позволяет приложению находить автоматически сгенерированные ресурсы, такие как CSS для определенных частей приложения. Кроме того, фреймворк может генерировать теги ссылок в элементе <head> страницы для предварительной загрузки ресурсов. Во время тестирования разработки Visual Studio и использования Hot Reload информация о целостности удаляется из ресурсов, чтобы избежать проблем при изменении файла во время работы приложения, и статические ресурсы не кэшируются, чтобы браузер всегда получал текущий контент.

Статические файлы и MapStaticAssets

При этом следует отметить, что MapStaticAssets не поддерживает ряд функций, которые поддерживаются UseStaticFiles, в частности:

- Обслуживание файлов с диска или встроенных ресурсов или других расположений
- Обслуживание файлов за пределами корневого каталога wwwroot
- Установка заголовков ответа HTTP
- Просмотр каталогов
- Обслуживание документов по умолчанию
- FileExtensionContentTypeProvider
- Обслуживание файлов из нескольких расположений

Таким образом, у разработчика начиная с ASP.NET Core 9 есть выбор, что использовать - **MapStaticAssets** или **UseStaticFiles**.

Статические файлы и MapStaticAssets

возможные оптимизации с MapStaticAssets включают:

- Обслуживание определенного ресурса один раз, пока файл не изменится или браузер не очистит свой кэш. Установка заголовков ETag и Last-Modified.
- Предотвращение использования браузером старых или устаревших ресурсов после обновления приложения. Установка заголовка Last-Modified.
- Настройка правильных заголовков кэширования.
- Использование кэширования.
- Обслуживание сжатых версий ресурсов, когда это возможно. Эта оптимизация не включает минимизацию.
- Использование CDN для обслуживания ресурсов ближе к пользователю.
- Снятие отпечатков ресурсов для предотвращения повторного использования старых версий файлов.

Применение MapStaticAssets

Обслуживаемые компонентом MapStaticAssets статические файлы хранятся в корневом веб-каталоге проекта, который по умолчанию представляет папку **wwwroot**, но его можно изменить с помощью метода **UseWebRoot**.

Допустим, у нас в проекте есть папка **wwwroot** со следующей структурой:

- Файл **index.html**
- Папка **js**
 - Файл **app.js**

Пусть в файле **index.html** будет какой-нибудь простенький контент с подключением файла **js/app.js**

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Test</title>
</head>
<body>
  <h1>Index Page</h1>
  <script src="js/app.js"></script>
</body>
</html>
```

А в файле **js/app.js** определим для теста какой-нибудь простенький скрипт:

```
console.log("Hello World");
```

В главный файл программы **Program.cs** добавим вызов MapStaticAssets

```
var builder =
WebApplication.CreateBuilder(args);
var app = builder.Build();
```

```
app.MapStaticAssets(); // обработка
статических файлов
```

```
app.MapGet("/", () => "Hello World!");
```

```
app.Run();
```